

Exploring and Testing APIs with Postman

Project Overview

This project demonstrates how to explore, authenticate, test, and automate API requests using Postman.

The DummyJSON API was selected for this project because it provides authentication and product endpoints that allow different HTTP methods to be tested.

The project demonstrates:

- API authentication
- GET requests
- POST requests
- PUT requests
- DELETE requests
- Postman environment variables
- Authentication token management
- Postman test scripts
- Pre-request scripts
- Collection Runner automation
- API Used

API: DummyJSON

Base URL: <https://dummyjson.com>

Documentation: <https://dummyjson.com/docs>

API Endpoints

Operation	HTTP Method	Endpoint	Purpose
Login	POST	/auth/login	Authenticate and obtain an access token
Get Products	GET	/products	Retrieve products
Create Product	POST	/products/add	Create a simulated product
Update Product	PUT	/products/1	Update product ID 1
Delete Product	DELETE	/products/1	Delete product ID 1

Authentication

Authentication is performed using the DummyJSON login endpoint:

POST <https://dummyjson.com/auth/login>

The login request uses the following credentials provided by the DummyJSON documentation:

```
{
  "username": "emilys",
  "password": "emilyspass"
}
```

A successful login returns an access token.

The token is automatically stored in the Postman environment using a Post-response script:

```
const jsonData = pm.response.json();

pm.environment.set("auth_token", jsonData.accessToken);
```

The token is then used by protected requests through Bearer Token authentication.

Postman Environment

The project uses an environment called:

DummyJSON Environment

The main variables are:

Variable	Purpose
base_url	Stores the API base URL
auth_token	Stores the authentication token
resource_id	Stores the product ID used for PUT and DELETE

Example:

```
base_url = https://dummyjson.com
resource_id =
```

CRUD Operations

1. GET: Retrieve Products

GET {{base_url}}/products

This request retrieves a list of products from the API.

Tests were added to verify:

The response status is 200

The response contains products

2. POST: Create Product

POST {{base_url}}/products/add

Example request body:

```
{
  "title": "Student Backpack",
  "description": "A backpack created for the API assignment",
  "price": 50,
  "stock": 20
}
```

Tests were added to verify:

The response status is 200

The response contains a product ID

3. PUT: Update Product

PUT {{base_url}}/products/{{resource_id}}

The environment variable resource_id is set to 1.

Example request body:

```
{
  "title": "Updated Student Backpack",
  "price": 65
}
```

Tests were added to verify:

The response status is 200

The response contains a product ID

4. DELETE: Delete Product

DELETE {{base_url}}/products/{{resource_id}}

The request uses product ID 1.

Tests were added to verify:

The response status is 200

The response contains isDeleted: true

Automated Token Handling

A Pre-request Script was used to demonstrate automatic token retrieval before a protected request.

The script sends a login request and stores the returned access token in the environment.

```
pm.sendRequest({
  url: pm.environment.get("base_url") + "/auth/login",
  method: "POST",
  header: {
    "Content-Type": "application/json"
  },
  body: {
    mode: "raw",
    raw: JSON.stringify({
      username: "emilys",
      password: "emilypass"
    })
  }
}, function (err, response) {
  if (err) {
    console.log(err);
    return;
  }

  const jsonData = response.json();
  pm.environment.set("auth_token", jsonData.accessToken);
});
```

This demonstrates how Postman can automatically obtain and store an authentication token instead of requiring it to be entered manually.

Automated Testing

Postman test scripts were used to verify API responses.

For example:

```
pm.test("Status code is 200", function () {
  pm.response.to.have.status(200);
});
```

Additional tests check important response properties, such as:

Product IDs

Product data
isDeleted status
Collection Runner

The Postman Collection Runner was used to execute multiple requests automatically.

The collection contains:

Login
Get Products
Create Product
Update Product
Delete Product

The Collection Runner was configured to use:

Collection: DummyJSON API Assignment
Environment: DummyJSON Environment
Iterations: 1

The automated run was successfully tested using Postman's Collection Runner.

Important Note About DummyJSON

DummyJSON simulates product modification operations.

Therefore, product creation, updating, and deletion are useful for demonstrating API requests and responses, but the changes are not treated as permanent database changes.

For this project, product ID 1 was used for the PUT and DELETE demonstrations because it is an existing product resource.

Project Files

The repository contains the following files:

DummyJSON-Postman-Assignment/
|
|— README.md
|— DummyJSON API Assignment.postman_collection.json
|— DummyJSON Environment.postman_environment.json
|— API Documentation / Report

The exported Postman collection contains all the API requests, tests, and scripts used in this project.

The environment file contains the variables required to run the collection.

Note: Any real authentication token should be removed from the environment file before uploading it to GitHub.

Learning Outcomes

Through this project, I learned how to:

- Understand API endpoints
- Work with HTTP methods
- Authenticate with an API
- Use Bearer Token authentication
- Create and use Postman environment variables
- Write Postman test scripts
- Use Pre-request Scripts
- Automate API requests
- Use the Collection Runner
- Troubleshoot API errors such as 404 Not Found

Author

Kalungi Bright Angel